

תכנון ארכיטקטורת התוכנה – מערכת ניהול כוח אדם ומשמרות ("המשבצת")

סטטוס: סופי (הוגש בתאריך 2026-09-06). מסמך זה בונה על docs/product-spec.md ועל ההחלטות המתועדות ב- CLAUDE.md. זהו תכנון מערכת ברמה גבוהה (תוצר קורס #3) – מבנה תיקיות, חתימות API מדויקות, ופרטי ממשק משתמש נמצאים במסמך התכנון הטכני המפורט (תוצר #4), לא כאן.

אילו רכיבים יהיו במערכת

- אפליקציית **Next.js (TypeScript)** – אפליקציית App Router יחידה המגישה גם את חוויית המנהל וגם את חוויית העובד. אין קוד/repo נפרד ל-frontend מול Server Components ו-Server Actions של Next.js הם ה-frontend.
- Supabase Postgres** – מסד נתונים.
- Supabase Auth** – אימות משתמשים (הנפקת session/JWT), בשימוש הן על ידי שרת ה-Next.js (דרך @supabase/ssr) והן נאסף שוב ברמת מסד הנתונים דרך Row-Level Security.
- Vercel** – מארח ופורס את אפליקציית ה-Next.js.

אין שירות API נפרד מסוג REST/GraphQL ואין מריץ משימות רקע (background job runner). שני אלה נמנעו במכוון – ראו "Server Actions מול API Routes" ו"חישוב תוקף הסמכות" בהמשך להסבר למה, ולמה זו בחירה סבירה בהיקף הפרויקט הזה (ראו גם את מסמך הסקייל).

האם משתמשים במסד נתונים

כן – Supabase Postgres. מסד נתונים יחסי מתאים היטב לתחום הזה: הסמכות, תפקידים, משמרות ושיבוצים הם כולם ישויות עם קשרים ברורים ביניהן (משמרת דורשת תפקידים; תפקיד דורש הסמכות; עובד מחזיק הסמכות) – בדיוק למה סכמה יחסית עם מפתחות זרים נועדה, בניגוד ל-document store.

אילו טבלאות/ישויות מרכזיות יהיו במסד הנתונים

טבלה	תפקיד	שדות מרכזיים
profiles	שורה אחת לכל משתמש (מרחיבה את auth.users של Supabase)	id, full_name, role (admin worker)
qualifications	סוגי הסמכה המוגדרים על ידי המנהל	id, name, renewal_interval_days (ניתן ל-null – לא פג לעולם)
qualification_options	ערכים אפשריים לבחירה עבור הסמכה (למשל	id, qualification_id, label, sort_order

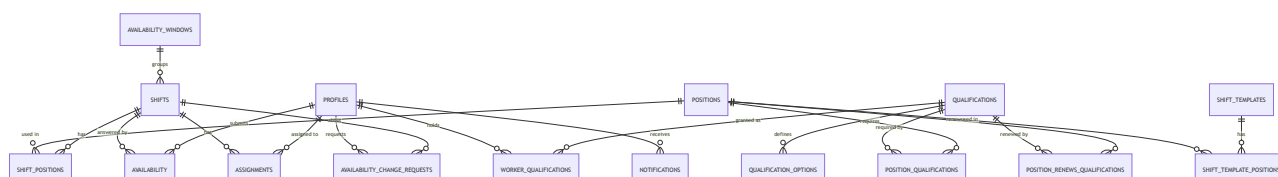
טבלה	תפקיד	שדות מרכזיים
	"דרגה" → "סג"ם"/"סג"ן"/"סר"ן"	
positions	סוגי תפקידים המוגדרים על ידי המנהל	id, name
position_qualifications	אילו הסמכות (ואופציונלית – אילו ערך ספציפי שלהן) תפקיד דורש	position_id, qualification_id, option_id (ניתן ל-null)
position_renews_qualifications	אילו הסמכה/ות מילוי התפקיד מחדש	position_id, qualification_id
worker_qualifications	אילו הסמכות מחזיק עובד	id, worker_id, qualification_id, option_id, source (self_reported admin_granted), status (pending approved rejected), obtained_at, reviewed_by, reviewed_at
shift_templates	חבילות שם-ולתפקידים הניתנות לשימוש חוזר	id, name
shift_template_positions	תפקידים + כמות עבור תבנית	template_id, position_id, headcount_needed
availability_windows	תקופה שבה עובדים מגישים זמינות	id, label, opens_at, closes_at
shifts	משמרת/תורנות קונקרטית	id, date, start_time, end_time, location, name (ניתן ל-null), availability_window_id (FK), published_at (ניתן ל-null)
shift_positions	תפקידים + כמות נדרשת למשמרת ספציפית	shift_id, position_id, headcount_needed
availability	תגובת עובד למשמרת אחת	id, worker_id, shift_id, is_available, responded_at
assignments	עובד ששובץ לתפקיד במשמרת	shift_id + position_id (מפתח זר → shift_positions), worker_id, created_by (ניתן ל-null) null = המנוע, מוגדר = המנהל
notifications	התראות בתוך האפליקציה לעובדים	id, worker_id, shift_id message, created_at, (null read_at (ניתן ל-null))
scheduling_constraints	פרמטרים הניתנים לכיוון על ידי המנהל עבור	id, type, qualification_option_id (ניתן ל-null)

טבלה	תפקיד	שדות מרכזיים
	ההיוריסטיקה (למשל מנוחה מינימלית), כולל חריגות אופציונליות לפי קטגוריית עובד	<code>enabled</code> , <code>null — null value</code> (ברירת מחדל), <code>value</code>
<code>worker_pairing_preferences</code>	אילו זוגות עובדים להימנע/להעדיף לשבץ יחד	<code>id</code> , <code>worker_id_1</code> , <code>worker_id_2</code> , <code>preference</code> (<code>avoid</code> / <code>prefer_avoid</code> / <code>prefer</code>)
<code>availability_change_requests</code>	בקשת עובד לשינוי לאחר סגירת חלון זמינות	<code>id</code> , <code>worker_id</code> , <code>shift_id</code> , <code>message</code> (ניתן ל- <code>null</code>), <code>acknowledged_at</code> , <code>acknowledged_by</code>
<code>app_settings</code>	הגדרות אפליקציה כלליות (טבלת singleton — שורה אחת בלבד)	<code>id</code> , <code>expiring_soon_days</code>

עקרונות מפתח מאחורי הסכמה:

- `position_renews_qualifications` היא טבלת קישור בפני עצמה (תפקיד → מספר הסמכות שהוא יכול לחדש), במקביל בדיוק ל- `position_qualifications` — תפקיד יכול לחדש יותר מהסמכה אחת בו-זמנית.
- **תוקף ההסמכה של עובד מחושב, לא נשמר.** אין עמודת `expires_at` בטבלת `worker_qualifications`. במקום זאת, התוקף נגזר בזמן השאילתה: `max(obtained_at, ...)` + (המשמרות שהושלמו שבהן העובד מילא תפקיד שמחדש הסמכה זו `qualification.renewal_interval_days` "הושלמה" משמעה ש- `shift.date` כבר חלף. זה חוסך צורך ב-job רקע או טריגר מתוזמן כדי לשמור ערך מאוחסן מסונכרן — אין רגע שבו כתיבה צריכה לקרות "כי הזמן עבר"; הערך פשוט מחושב בכל קריאה. בהיקף הפרויקט הזה (עשרות-מאות שורות) זה פשוט ומהיר; זו בדיוק הנקודה לחזור אליה אם אי פעם יהיה צורך בסקייל גדול משמעותית יותר.
- ל- `assignments` אין עמודת `status` נפרדת. האם שיבוץ הוא "מוצע" או "סופי" נגזר מ- `published_at` של המשמרת שלו: `NULL` → עדיין בבחינת המנהל (עובדים אסור שיראו אותו); מוגדר → פורסם וגלוי לעובד המשובץ. פעולת פרסום אחת קובעת `published_at` על כל המשמרות בחלון זמינות בבת אחת ויוצרת את שורות ה- `notifications` המתאימות.

קשרי ישויות



אילו עמודים יהיו באפליקציה

משותף:

- `/login` — כניסה בדוא"ל+סיסמה (ראו "אימות והרשאות" בהמשך למה נבחר מודל זה ולא כניסה חסרת סיסמה).
- `/dashboard` — נתיב יחיד, **מודע-תפקיד**: מציג את דשבורד המנהל או דשבורד העובד לפי `profiles.role`. מודל מנטלי אחד של "בית" במקום שני נתיבים מנותקים; זהו מסך הבית המוגדר כברירת מחדל — אינו מופיע בתפריט הניווט (מגיעים אליו דרך הלחיצה על הלוגו), אלא הנתיב עצמו.

מנהל בלבד (`role = admin`, נאכף על ידי RLS + שומר ברמת ה-`layout`):

- `CRUD` — `/admin/qualifications` הסמכות (שם, תדירות חידוש, ערכים אפשריים).
- `CRUD` — `/admin/positions` סוגי תפקידים; קביעת הסמכות נדרשות ואילו הסמכה/ות התפקיד מחדש.
- `/admin/personnel` ו-`/admin/personnel/[workerId]` — רשימת עובדים; מסך פרטים לכל עובד/ת עם הסמכותיו/ה, המאפשר למנהל לאשר/לדחות דיווחים עצמיים ממתינים או להעניק/לבטל הסמכות ישירות.
- `CRUD` — `/admin/shift-templates` תבניות משמרת.
- `/admin/shifts` (+ `/admin/shifts/past`) לתצוגת היסטוריה, ותצוגת יומן חודשי/שבועי לצד רשימה) — יצירה/עריכה/מחיקה של משמרות (אופציונלית החל מתבנית), מקובצות לפי חלון זמינות.
- `/admin/availability-windows/[id]` — פתיחה/סגירה של חלון זמינות; צפייה בזמינות שהוגשה לכל משמרת.
- `/admin/schedule/[windowId]` — הפעלת יצירת לוח משמרות עבור החלון, בחינה/עריכה של השיבוצים המוצעים, פתרון משמרות תת-מאוישות מסומנות וקונפליקטים רכים של העדפות זיווג, פרסום.
- `/admin/settings` — עמוד הגדרות מאוחד: הפעלה/כיוון אילוצי שיבוץ (כולל חריגות לפי קטגוריית עובד), ניהול העדפות זיווג עובדים, והגדרות אפליקציה כלליות (כמו `expiring_soon_days`) כמדורים נוספים — עמוד אחד, לא עמוד לכל נושא.

עובד בלבד (`role = worker`):

- `/my-qualifications` — צפייה בהסמכות של העובד/ת עם סטטוס תוקף מחושב; דיווח עצמי על הסמכה חדשה (נכנס כ- `pending`).
- `/availability` — מענה למשמרות בחלון הזמינות הפתוח כרגע; לאחר סגירת חלון, צפייה בתגובות קודמות (`read-only`) עם אפשרות לבקש שינוי.
- `/my-shifts` — משמרות שפורסמו והעובד/ת משובץ/ת אליהן.
- `/notifications` — התראות בתוך האפליקציה, כולל סימון כנקרא.

Server Actions מול API Routes

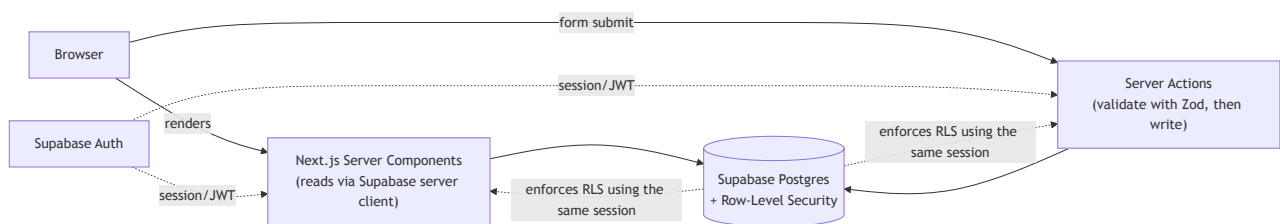
החלטה: Next.js Server Actions לכל הפעולות המשנות נתונים בפועל; ללא שכבת REST API נפרדת.

לאפליקציה הזו יש לקוח יחיד (ה-`frontend` שלה עצמה) — אין אפליקציית מובייל, שותף חיצוני, או API ציבורי לשרת, כך ששכבת `REST/API routes` קונבנציונלית תהיה רק שכבת תיווך ללא צרכן. `Server Actions` רצות עם ה-`session` של המשתמש המבקש, כך ש-`Row-Level Security` חל בדיוק כאילו הקריאה הגיעה ישירות מהדפדפן. `Route Handlers` (`API routes`) `Route Handlers` ייכנסו לתמונה רק אם משהו שלא מופעל על ידי אינטראקציית משתמש יזדקק לנקודת קצה HTTP (למשל `webhook`) — שום דבר בתכנון הנוכחי לא זקוק לכזה.

`Server Actions` מייצגות (מודול אחד לכל `domain`, לפי עקרון הארגון ב-`CLAUDE.md` — לא רשימה ממצה):

- `features/qualifications/actions.ts`: `createQualification`, `updateQualification`, `deleteQualification`
- `features/positions/actions.ts`: `createPosition`, `updatePosition`, `setPositionRequirements`
- `features/worker-qualifications/actions.ts`: `selfReportQualification`, `grantQualification`, `revokeQualification`, `reviewQualification` (אישור/דחייה)
- `features/shift-templates/actions.ts`: `createTemplate`, `updateTemplate`, `deleteTemplate`
- `features/shifts/actions.ts`: `createShift` (אופציונלית, `fromTemplateId`), `updateShift`, `deleteShift`
- `features/availability/actions.ts`: `submitAvailability`, `requestAvailabilityChange`
- `features/scheduling/actions.ts`: `generateSchedule` (מריצה את ההיוריסטיקה, כותבת ל-`assignments`), `addAssignment` / `removeAssignment`, `publishShift` / `publishAllShiftsInWindow`

איך המידע זורם בין ה-Frontend, ה-Backend, וה-Database



תרשים: הדפדפן קורא נתונים דרך `Server Components` ושולח טפסים ל-`Server Actions`; שניהם פונים ל-`Supabase Postgres` תחת אותו `session`, כאשר `Row-Level Security` נאכפת בצד ה-DB על סמך אותו `session` בדיוק.

- **קריאה:** `Server Components` שולפות נתונים ישירות מ-`Supabase` באמצעות ה-`session` של המשתמש המחובר (דרך עוגיות `@supabase/ssr`) — כך ששאלתה מחזירה רק את מה ש-`RLS` מתיר לאותו משתמש, בלי שהעמוד יצטרך לממש את הסינון הזה בעצמו.
- **כתיבה:** טפסים קוראים ל-`Server Actions`, שמאמתות קלט עם `Zod` ואז כותבות ל-`Supabase` באמצעות אותו `session` — `RLS` היא בדיקת ההרשאה הסופית גם אם בדיקה ברמת הממשק הוחמצה (הגנה בשכבות, לא הבדיקה היחידה — `Server Actions` גם בודקות מחדש תפקיד לפני ביצוע כתיבות שמורות-מנהל).
- **אין קריאות `Supabase` בצד לקוח עם הרשאות מוגברות** — שום דבר לא משתמש במפתח ה-`service role` של `Supabase` מהדפדפן; רק `Server Components/Actions` רצות עם ההקשר הדרוש מעבר ל-`session` רגיל, ורק היכן שבאמת נדרש (למשל יצירת חשבון עובד/ת חדש/ה).

אילו משתמשים והרשאות יהיו במערכת

שני תפקידים, ללא שכבת ריבוי-ארגונים (ראו `CLAUDE.md`):

משאב	מנהל	עובד
הסמכות, תפקידים, תבניות משמרת	CRUD מלא	קריאה בלבד
משמרות, חלונות זמינות	CRUD מלא	קריאה בלבד (משמרות רלוונטיות משלו/ה)
פרופיל עצמי ו- <code>worker_qualifications</code> עצמי	מלא (כמנהל על כולם)	קריאה של עצמו/ה; יכול/ה להכניס דיווח עצמי <code>pending</code>
אישור/הענקת הסמכות	כן	לא
תגובות זמינות	קריאת הכול	קריאה/כתיבה של עצמו/ה בלבד
שיבוצים לפני פרסום (<code>published_at IS NULL</code>)	קריאה/כתיבה	ללא גישה כלל — אסור לראות לוי"ז מוצע לפני שהוא סופי
שיבוצים אחרי פרסום	קריאת הכול	קריאה של עצמו/ה בלבד
התראות	—	קריאה של עצמו/ה בלבד

נאכף בשתי שכבות:

1. **מדיניות Row-Level Security** ב-Postgres — גבול ההרשאה האמיתי; גם ממשק פגום או פרוץ לא יכול לקרוא/לכתוב נתונים שהמדיניות אוסרת.
2. **בדיקת תפקיד ברמת ה-Server Action** — בדיקת הגנה-בשכבות לפני ביצוע כתיבות שמורות-מנהל, כך שטעות מתבטאת כדחייה מפורשת של הפעולה ולא מסתמכת רק על כך שמסד הנתונים יחזיר בשקט אפס שורות.

הכלל "עובד לא יכול לראות שיבוצים לא-מפורסמים" הוא כלל ההרשאה היחיד במערכת הזו שקשור ל**תזמון**, לא רק לתפקיד — שווה להדגיש כי קל לפספס אותו כשכותבים את מדיניות ה-RLS (מדיניות נאיבית "עובדים יכולים לקרוא שיבוצים שבהם `worker_id = auth.uid()`" הייתה מדליפה את הלוי"ז המוצע מוקדם מדי).

אימות משתמשים (Authentication)

החלטה: דוא"ל+סיסמה, חשבונות שנוצרים על ידי המנהל, ללא שליחת אימייל אמיתית בגרסה 1. הרשמה עצמית ציבורית מבוטלת ברמת פרויקט ה-Supabase. המנהל יוצר חשבון עובד/ת ישירות (דוא"ל + סיסמה, `email_confirm: true`) דרך ה-API של Admin של Supabase — דורש את מפתח ה-service-role בצד שרת בלבד. לא נשלח מייל אישור/הזמנה; המנהל מעביר את פרטי ההתחברות מחוץ למערכת. נשקלו ונדחו במפורש עבור גרסה 1: כניסה מבוססת magic-link/OTP (שליחת המייל המובנית של Supabase מוגבלת בקצב של כמה בשעה — סיכון אמיתי בהדגמה חיה) ובניית הזמנות מייל אמיתיות עכשיו (תוספת פשוטה ומבודדת להמשך — החלפת פונקציה אחת, בלי לשנות שום דבר אחר).

אילו ספריות או שירותים חיצוניים משולבים, ולמה

ספרייה/שירות	למה
Supabase (@supabase/supabase-js , @supabase/ssr	נדרש על ידי המטלה; מספק Postgres + Auth + RLS בשירות אחד.
Vercel	נדרש על ידי המטלה לפריסה.
Tailwind CSS + shadcn/ui (מבוסס Base UI	קוד הרכיבים נוצר ישירות ל-repo במקום להיכנס כתלות אטומה — מתאים לדרישה להבין ולהסביר כל חלק בקוד, ועדיין הרבה יותר מהיר מבניית רכיבים נגשים מאפס. גם הסיבה שהעיצוב הרספונסיבי מובנה מהיום הראשון (ראו "עיצוב רספונסיבי" בהמשך).
Zod	אימות קלט לכל Server Action — תשובה ברורה וניתנת להסבר לדרישת "אימות קלט" של המטלה, ומשתלב באופן טבעי עם טיפוס TypeScript.

הוסר מהתכנון המקורי, בכוונה, אחרי שהתברר שלא נדרש בפועל: date-fns — התכנון המקורי הניח שתידרש ספריית תאריכים חיצונית לחישוב תוקף/חידוש הסמכות, אך בפועל חישובי התאריכים (computeExpiresOn ב-features/worker-qualifications/queries.ts של JavaScript, ללא תלות נוספת — פחות קוד לתחזק, ומכוסה ישירות בבדיקות יחידה.

לא בשימוש, במכוון: שירות job/cron רקע (תוקף הסמכה מחושב בזמן קריאה, לא נכתב על ידי משימה מתוזמנת — ראו למעלה), ספק אימייל/SMS נפרד (התראות בתוך האפליקציה בלבד בגרסה 1), ספריית constraint-solver (מנוע ההתאמה הוא היוריסטיקה כתובה-יד, במכוון, כדי שתישאר ניתנת להסבר).

עיצוב רספונסיבי

מובנה מהתחלה בכל עמוד, לא מטופל כגרסת מובייל נפרדת או כעבודה נדחית — utilities מונחי-מובייל של Tailwind ורכיבי shadcn/ui הרספונסיביים כברירת מחדל הופכים את זה כמעט לחינם, וטיפול בכך בדיעבד היה אומר לבנות מחדש פריסות שכבר קיימות במקום זאת.

- עמודי עובד (/my-shifts , /availability , /my-qualifications , דשבורד עובד) הם mobile-first: בדיקת המשמרת הקרובה או הגשת זמינות היא אינטראקציית "בדיקה מהירה בטלפון" קלאסית.
- עמודי מנהל, בייחוד מסך בחינה/עריכה של לוח המשמרות, נשארים רספונסיביים במלואם אך מותאמים לדסקטופ: בחינה של רשת משמרות × תפקידים × עובדים היא במהותה עתירת מידע ואכן שמישה יותר על מסך גדול. זהו טרייד-אוף רגיל ומכוון עבור כלי ניהול, לא קיצור דרך — העמודים עדיין עובדים במובייל, פשוט לא תוכננו מובייל-תחילה.